

# From Prediction to Recommendation: A Reproducibility Study of Honest Evaluation in Deep Knowledge Tracing

**Editors:** Ignacio Sanhueza

## Abstract

We present a reproducibility study of three deep knowledge tracing models (DKT, SAKT, and AKT) using the pyKT toolkit, run on CPU using published hyperparameters, and we connect the question of honest evaluation to downstream exercise recommendation. On the standard ASSISTments2009 benchmark we reproduce the published ordering  $AKT > DKT > SAKT$  under pyKT’s leakage free, question level protocol. Replicating on a subsample of EdNet-KT1, we find that removing multi-concept label leakage lowers DKT’s AUC by 0.07 on ASSISTments2009 but by 0.31 on EdNet, whose questions carry up to seven concepts. Under honest evaluation none of the deep models beats a trivial question-difficulty baseline on EdNet. Finally, we add a Zone of Proximal Development recommendation layer that places students in a target difficulty band far more reliably than naive policies, and we validate it with a calibration analysis. Our results show that the **evaluation protocol is a precondition for trustworthy recommendation**, and that its impact grows with concept density.

**Keywords:** Knowledge Tracing, Deep Learning, Reproducibility, Exercise recommendation

## 1. Introduction

Knowledge Tracing (KT) models a student’s evolving mastery from their history of correct and incorrect answers, predicting the probability that they will answer the next exercise correctly [Feng et al. \(2009\)](#); [Piech et al. \(2015\)](#). This estimated knowledge state is the natural substrate for personalization in intelligent tutoring systems, since once we can predict how a student will perform, we can decide what they should practice next. Deep Knowledge Tracing (DLKT) replaced hand-crafted models with neural sequence models (recurrent [Piech et al. \(2015\)](#), self-attentive [Pandey and Karypis \(2019\)](#), and context-aware attentive [Ghosh et al. \(2020\)](#)) reporting steady gains on standard benchmarks.

However, two assumptions behind these reported gains are rarely examined together. First, the field has an evaluation and reproducibility problem. [Liu et al. \(2022\)](#) show that many DLKT results are inflated by *label leakage*: when a single exercise is annotated with several knowledge concepts, the common practice of expanding it into one interaction per concept lets the model “predict” repeated copies of the same answer seen previously, scoring the model at the concept level rather than at the question level. Under a leakage free, question level protocol, several reported improvements shrink. Also, even when these probabilities are used for pedagogical decisions (for instance, selecting the next exercise in an optimal difficulty band) it is seldom verified that they are calibrated, i.e., that a predicted 0.6 corresponds to a real 60% success rate. Meaning a recommendation policy is only as trustworthy as the probability it targets.

In this work we conduct a reproducibility study of three representative DLKT models, which are DKT, SAKT, and AKT, using the pyKT toolkit [Liu et al. \(2022\)](#), then link the evaluation question to the downstream decision. We reproduce results on the standard ASSISTments2009 dataset [Feng et al. \(2009\)](#) and replicate them on a subsample of EdNet-KT1, a dataset not used in the original papers of these models. We add two trivial baselines: a constant global-mean predictor and a per-question difficulty predictor, and, as an extension, a Zone of Proximal Development (ZPD) exercise selection layer that we validate through a calibration analysis. All experiments run on a single CPU, utilizing published hyperparameters and single runs, reflecting a realistic low-resource reproduction setting.

We define the study around the following questions:

- Can the reported results of DKT, SAKT, and AKT be reproduced independently, under the honest question-level protocol?
- How much does multi-concept label leakage inflate the naive concept-level evaluation, and does this effect generalize to a second dataset?
- Are the reproduced probabilities calibrated enough to support a difficulty-targeting recommendation policy?

The main findings and contributions found were:

- We reproduce the published ordering  $AKT > DKT > SAKT$  on ASSISTments2009 under question level evaluation, with AUC values within pyKT’s expected reported range, confirming that the toolkit is replicable in an independent CPU only environment.
- We quantify the label leakage inflation and show it is far more severe on multiconcept data: DKT’s AUC drops from 0.83 (concept level) to 0.75 (question level) on ASSISTments2009, but from 0.93 to 0.61 on EdNet-KT1, whose questions carry up to seven concepts. Honest evaluation therefore is **not cosmetic**, it can change the apparent magnitude of the state of the art, and does so differently across datasets.
- We show that turning KT predictions into recommendations requires trustworthy probabilities, our ZPD policy places students in the target difficulty band far more consistently than naive policies, and a calibration analysis confirms that the targeted 0.6 corresponds to a real  $\sim 60\%$  success rate.
- We release [code](#) to reproduce every result and figure.

## 2. Related Work

Knowledge Tracing originates with Bayesian Knowledge Tracing [Corbett and Anderson](#), which tracks each skill as a latent variable. DKT [Piech et al. \(2015\)](#) recast the task as recurrent sequence modeling; SAKT [Pandey and Karypis \(2019\)](#) introduced self-attention over past interactions; and AKT [Ghosh et al. \(2020\)](#) added monotonic, context-aware attention with Rasch-model embeddings. These three span the dominant design choices (recurrence, self-attention, context-aware attention), which is why we reproduce them.

|                            | ASSISTments2009 | EdNet-KT1 (subsample) |
|----------------------------|-----------------|-----------------------|
| Students                   | 4,217           | 2,469                 |
| Interactions               | 282,619         | 334,896               |
| Questions                  | 17,737          | 11,672                |
| Knowledge concepts         | 123             | 188                   |
| Max. concepts per question | 4               | 7                     |

Table 1: Dataset statistics after preprocessing.

Reported KT gains are hard to compare because datasets, preprocessing, and metrics vary across papers. pyKT Liu et al. (2022) standardizes preprocessing and exposes a form of label leakage: when an exercise carries several concepts, expanding it into one interaction per concept and scoring at the concept level lets the model exploit repeated copies of the same response, inflating performance. pyKT advocates a question-level protocol that emits one prediction per exercise. We adopt this protocol and quantify how much leakage inflates the naive evaluation, focusing on datasets whose exercises carry many concepts.

KT is ultimately a means to choose what a student should study next. Work in intelligent tutoring systems selects exercises following Vygotsky’s Zone of Proximal Development Vygotsky (1978), the region of tasks that are neither trivial nor out of reach. Operationalizing it requires a target success probability, on which the literature does not agree: information is maximized at  $P = 0.5$ , whereas the “85% rule” Wilson et al. (2019) favors  $P \approx 0.85$  for learning speed. Rather than proposing a new policy, we adopt a moderate target and focus on validating that the model’s probabilities are reliable enough to hit it.

A predicted probability is actionable only if calibrated: among cases where the model outputs 0.6, about 60% should be positive. Neural networks are often miscalibrated Guo et al. (2017), and calibration, standard in classification but rarely reported in KT, is exactly what pedagogical decisions on top of KT depend on. We measure it with the Expected Calibration Error and check that the difficulty band our policy targets matches a real success rate.

### 3. Datasets

We evaluate on two public datasets: ASSISTments2009 as the primary benchmark, and a subsample of EdNet-KT1 for replication. Table 3 summarizes their statistics. Both are processed with pyKT’s standard pipeline: interactions are ordered by timestamp, sequences shorter than three interactions are discarded, longer sequences are split into windows of at most 200 interactions, and an 80/20 student level train/test split is applied, with a separate question level test set built for the honest evaluation protocol.

**ASSISTments2009:** Collected from the ASSISTments online tutoring platform, this is the standard KT benchmark. We use the *skill-builder* corrected and collapsed version. Each interaction records a (student, question, concept, correct/incorrect) tuple ordered in time.

**EdNet-KT1:** EdNet-KT1 Choi et al. (2020) is a large-scale log (roughly 95M interactions from 784k students) from the Santa English learning platform. Because the full dataset does not fit in memory on the hardware used, we use a deterministic subsample of

about 2,500 students (a fixed prefix of pyKT’s shuffled user order). Two properties make it a useful complement to ASSISTments2009. First, EdNet records `user_answer` as the chosen multiple-choice letter rather than a binary outcome, so correctness must be recovered by joining with the question metadata (`correct_answer`), which pyKT handles during preprocessing. Second, its questions are more densely multi-concept (up to 7 concepts per question, compared to 4 in ASSISTments2009), which, as we show in Section 6, makes it a more severe test of concept level label leakage.

## 4. Methodology

### 4.1. Models and reproduction protocol

We reproduce three DLKT models that span the dominant architectural families: DKT, an LSTM over the interaction sequence; SAKT, a single self-attention block that attends from the current question to past interactions; and AKT, a context-aware attention model with monotonic attention decay and Rasch-model embeddings. All models are taken from the pyKT toolkit Liu et al. (2022) and trained with PyTorch on CPU.

To keep the reproduction faithful and low-cost, we follow pyKT’s published hyperparameters and perform single runs rather than a hyperparameter search (no sweep). We use fold 0 of the 5-fold split for training and validation, optimize with Adam, and apply early stopping on the validation AUC (patience 10, maximum 200 epochs). The only exception is AKT on EdNet, whose cost on CPU forced a lower epoch cap. We discuss this limitation in Section 7. The exact hyperparameters are listed in Section 5.

### 4.2. Honest evaluation

Our headline metric follows pyKT’s question-level protocol, which avoids the label leakage of the naive concept-level evaluation. When a question is annotated with multiple concepts, pyKT expands it into one interaction per concept; scoring the model at that level lets it exploit repeated copies of the same response. The honest protocol instead emits a single prediction per question by fusing the model’s per concept predictions. We report the standard *late-mean* fusion (the mean of the per concept probabilities) and, for models that also produce an early fusion (*early preds*, available for SAKT and AKT), that variant as the headline. We report both AUC and accuracy but treat AUC as primary: it measures ranking quality and is robust to the class imbalance of KT data, whereas accuracy can be inflated by predicting the majority class, as our global-mean baseline illustrates.

### 4.3. Trivial baselines

Beyond the three DLKT models, we add two non-temporal baselines evaluated on the same question-level test set. The *global-mean* predictor outputs a constant equal to the average correctness rate in the training set; it cannot rank and defines the AUC floor of 0.5. The *question-difficulty* predictor outputs, for each question, its average correctness in the training set (falling back to the global mean for unseen questions) it is the KT analogue of a Most-Popular baseline and captures item difficulty without modeling the student. A DLKT model earns its complexity only if it beats question difficulty, since surpassing it requires modeling the student’s evolving state rather than the item alone.

#### 4.4. From prediction to recommendation

A KT model predicts but does not decide. As an extension, we add a recommendation layer that selects the next exercise. After a student’s history, DKT outputs a probability of success for each of the  $C$  concepts in a single pass, this vector is the estimated difficulty of practicing each concept next. Following the Zone of Proximal Development, we recommend the concept whose predicted success probability is closest to a moderate target  $\tau = 0.6$ :

$$c^* = \arg \min_{c \in \{1, \dots, C\}} |\hat{P}(\text{correct} \mid c) - \tau|. \quad (1)$$

We treat  $\tau$  as a design parameter and report a sensitivity analysis over  $\{0.5, 0.6, 0.7\}$  in Section 5. Since we have no live students, we evaluate the policy offline by how well it places students in the target difficulty band: the fraction of recommendations with predicted success in  $[0.5, 0.7]$  and the mean distance to  $\tau$ , comparing ZPD against random, hardest, and easiest policies.

#### 4.5. Calibration

The ZPD target is meaningful only if the model’s probabilities are calibrated. We assess calibration on the honest predictions with a reliability diagram and the Expected Calibration Error (ECE), which bins predictions by confidence and averages the gap between predicted probability and observed accuracy. We then check the observed success rate inside the ZPD band, that is if a predicted 0.6 corresponds to a real success rate near 0.6, the recommendation targets a genuine difficulty rather than an artifact of the model.

### 5. Parameter Analysis

We use pyKT’s published hyperparameters for every model, without any tuning on our side (Table 5). All models are trained with Adam; DKT and SAKT use a learning rate of  $10^{-3}$  and batch size 64, while AKT follows its published setting of a lower learning rate ( $10^{-4}$ ) and a smaller batch size (32).

| Model | Architecture-specific                                                       | LR        | Batch | Seed |
|-------|-----------------------------------------------------------------------------|-----------|-------|------|
| DKT   | emb. size 200; dropout 0.2                                                  | $10^{-3}$ | 64    | 42   |
| SAKT  | emb. size 256; 8 heads; 1 block; dropout 0.2; seq. len 200                  | $10^{-3}$ | 64    | 42   |
| AKT   | $d_{\text{model}}$ 256; $d_{\text{ff}}$ 512; 4 blocks; 8 heads; dropout 0.2 | $10^{-4}$ | 32    | 3407 |

Table 2: Model hyperparameters used by pyKT

We cap training at 200 epochs and stop early when validation AUC does not improve for 10 epochs, using fold 0. On ASSISTments2009, DKT and SAKT reach their best validation AUC at epochs 4 and 6 respectively; on EdNet, at epochs 6 and 14. AKT is far more expensive on CPU: its training did not complete within our compute budget on either dataset, so we evaluate the best checkpoint saved up to the point of interruption. This compute limitation is discussed in Section 7.

The ZPD target  $\tau$  is a design choice, which spans a range in the literature (information is maximized at  $P = 0.5$ , while the “85% rule” [Wilson et al. \(2019\)](#) favors  $P \approx 0.85$ ). We therefore treat  $\tau$  as a parameter and verify that the policy is not sensitive to its exact value. Table 5 shows that, on DKT/ASSISTments2009, the recommended exercise lands at a predicted success probability almost exactly equal to  $\tau$  for every target in  $\{0.5, 0.6, 0.7\}$ , with essentially all recommendations falling within  $\tau \pm 0.1$ . The mechanism is robust;  $\tau = 0.6$  is a reasonable moderate choice, not a tuned value.

| Target $\tau$ | Mean predicted $P$ | % in $[\tau - 0.1, \tau + 0.1]$ | Mean $ P - \tau $ |
|---------------|--------------------|---------------------------------|-------------------|
| 0.5           | 0.500              | 100.0                           | 0.002             |
| 0.6           | 0.600              | 100.0                           | 0.002             |
| 0.7           | 0.697              | 99.9                            | 0.006             |

Table 3: Sensitivity of the ZPD policy to the target  $\tau$

## 6. Results and Discussion

Table 4 reports the honest question-level AUC and accuracy for all models and baselines on both datasets.

Table 4: Question-level (honest) results. Best DLKT per dataset in bold.

| Method              | ASSISTments2009 |       | EdNet-KT1    |       |
|---------------------|-----------------|-------|--------------|-------|
|                     | AUC             | ACC   | AUC          | ACC   |
| AKT                 | <b>0.787</b>    | 0.741 | 0.609        | 0.657 |
| DKT                 | 0.749           | 0.720 | <b>0.610</b> | 0.643 |
| SAKT                | 0.729           | 0.709 | 0.603        | 0.653 |
| Question difficulty | 0.689           | 0.675 | 0.683        | 0.693 |
| Global mean         | 0.500           | 0.638 | 0.500        | 0.665 |

On ASSISTments2009 we reproduce the published ordering  $AKT > DKT > SAKT$ , with AUC values within the range reported by pyKT, and all three DLKT models clearly beat the trivial baselines. This confirms that pyKT is replicable in an independent, CPU-only environment using published hyperparameters and single runs.

The honest protocol changes the picture substantially, and unequally across datasets. Table ?? contrasts the concept-level AUC (the leaky metric) with the honest question-level AUC for DKT, both computed on the same test set. On ASSISTments2009 the drop is moderate (0.83 to 0.75), but on EdNet it is dramatic (0.93 to 0.61): a gap roughly four times larger. The reason is structural: EdNet questions carry up to seven concepts, so the concept-level expansion produces many repeated copies of the same response that the model trivially predicts, inflating the naive metric far more than on ASSISTments2009. On the same test set, DKT’s AUC falls from 0.82 (concept level) to 0.75 (question level) on ASSISTments2009, and from 0.92 to 0.61 on EdNet.

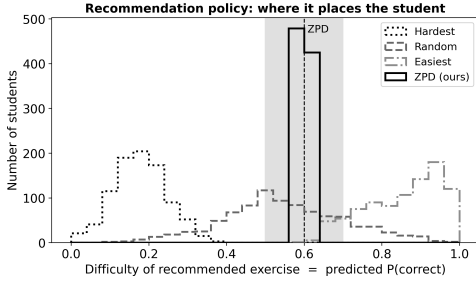


Figure 1: Difficulty of the recommended exercise under each policy

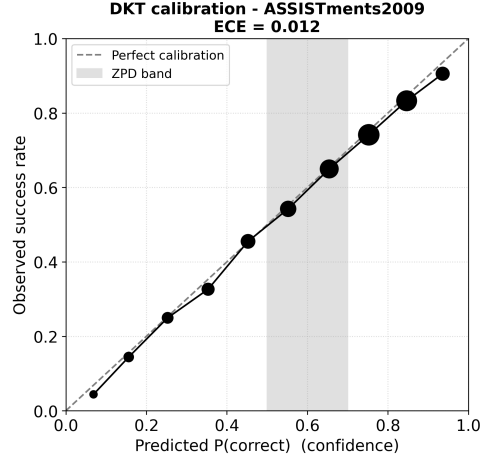


Figure 2: Reliability diagram for DKT (ECE = 0.012)

This inflation is not just cosmetic: it can reverse conclusions. On EdNet, the concept level metric suggests the deep models are far superior, yet under honest evaluation none of the three DLKT models (AUC  $\sim 0.61$ ) beats the trivial question-difficulty baseline (0.683). In other words, once leakage is removed, modeling the student’s temporal trajectory provides no advantage over simply knowing each item’s average difficulty on this dataset. We note that the deep models may be further penalized by our reduced 2,500-student subsample and single-fold setup; nonetheless, the qualitative reversal is a concrete example of why evaluation protocol matters. Consistent with this, the global-mean predictor attains an accuracy of 0.638-0.665 despite an AUC of 0.5, confirming that accuracy is misleading under class imbalance and that AUC is the appropriate headline metric.

Furthermore, we build the ZPD recommendation layer on DKT and evaluate how well each policy places students in the target difficulty band. The ZPD policy targets a predicted success probability of 0.6 and reaches it for 100% of students (mean predicted  $P = 0.60$ ), whereas a random next exercise policy lands in the  $[0.5, 0.7]$  band only 42.7% of the time (mean  $P = 0.55$ ). Naive alternatives systematically miss the band: always recommending the hardest available concept yields a mean  $P$  of 0.18 (frustrating), and the easiest yields 0.86 (boring). Figure 1 shows the resulting difficulty distributions.

Crucially, this targeting is only meaningful if the probabilities are calibrated. DKT on ASSISTments2009 is well calibrated, with an Expected Calibration Error of 0.012; inside the ZPD band the model predicts 0.61 and the observed success rate is 0.61 (Figure 2). The difficulty the policy targets is therefore real, not an artifact of the model: a predicted 0.6 corresponds to a genuine 60% chance of success. This closes the loop from reproduction, to recommendation, to validation

## 7. Conclusions and Future Work

We conducted a reproducibility study of three deep knowledge tracing models with pyKT, connecting the question of honest evaluation to the downstream task of exercise recommendation. Our findings answer the three research questions. The published results and the ordering  $AKT > DKT > SAKT$  reproduce on ASSISTments2009 under the honest question-level protocol, in an independent CPU only setting, confirming that pyKT is replicable. Removing multi concept label leakage lowers DKT’s AUC by 0.07 on ASSISTments2009 but by 0.31 on EdNet, whose questions carry more concepts; on EdNet the honest evaluation even reverses the ranking, with none of the deep models beating a trivial question difficulty baseline. Honest evaluation is therefore not a formality: it changes the apparent magnitude of the state of the art, and does so more severely on densely multi-concept data. On top of the reproduced model we built a ZPD recommendation layer and validated it: the policy places students in the target difficulty band far more reliably than naive alternatives, and a calibration analysis ( $ECE = 0.012$ ) confirms the targeted probability is real.

The broader message is that in knowledge tracing the evaluation protocol is not merely a leaderboard convention but a precondition for trustworthy decisions: a recommendation policy built on leaky, miscalibrated probabilities would steer students toward an illusory difficulty.

Our study has several limitations that also outline future work. All results use a single fold and single runs without error bars; running the full five folds would provide confidence intervals. AKT could not be trained to completion on EdNet within our CPU budget, so its EdNet result comes from an interrupted run and should be read with caution; a GPU would remove this constraint and allow a larger EdNet subsample. The recommendation is evaluated offline against the model’s own predictions, mitigated but not eliminated by the calibration check; measuring real learning gains would require a user study or an A/B test. Finally, our recommendation operates at the concept level via DKT; extending it to the question level, for instance with AKT, is a natural next step. Despite these limitations, the study shows that a faithful, honestly evaluated, and calibrated KT model is a sound basis for exercise recommendation, and that the honest protocol matters most exactly where deep models are most tempting to overtrust.

## References

- Youngduck Choi, Youngnam Lee, Dongmin Shin, Junghyun Cho, Seoyon Park, Seewoo Lee, Jineon Baek, Chan Bae, Byungsoo Kim, and Jaewe Heo. Ednet: A large-scale hierarchical dataset in education. In *International Conference on Artificial Intelligence in Education (AIED)*, pages 69–73. Springer, 2020.
- Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4).
- Mingyu Feng, Neil Heffernan, and Kenneth Koedinger. Addressing the assessment challenge with an online system that tutors as it assesses. *User Modeling and User-Adapted Interaction*, 19(3):243–266, 2009.



- Aritra Ghosh, Neil Heffernan, and Andrew S. Lan. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 2330–2339, 2020.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.
- Zitao Liu, Qiongqiong Liu, Jiahao Chen, Shuyan Huang, Jiliang Tang, and Weiqi Luo. pykt: A python library to benchmark deep learning based knowledge tracing models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.
- Shalini Pandey and George Karypis. A self-attentive model for knowledge tracing. In *Proceedings of the 12th International Conference on Educational Data Mining (EDM)*, pages 384–389, 2019.
- Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.
- Lev S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.
- Robert C. Wilson, Amitai Shenhav, Mark Straccia, and Jonathan D. Cohen. The eighty five percent rule for optimal learning. *Nature Communications*, 10(1):4646, 2019.